深圳大学实验报告

课程名称：Python 程序设计

项目名称：基于 NumPy 的图像特征提取

学院：人工智能学院

专业：人工智能（IEEE 荣誉班）

指导教师：舒婷

报告人：徐潇阁　学号：2025681120

实验时间：2026 年 11 月 10 日至 11 日

提交时间：2026 年 11 月 11 日

教务处制

1 Objectives：任务

本项目在给定 OpenCLIP ViT-B/32 图像权重的条件下，使用 NumPy 实现图像编码器前向计算，不进行模型训练。输入为预处理后的 float32 图像批次 $[B, 3, 224, 224]$；输出为每张图像一个单位长度的 512 维特征 $[B, 512]$。独立提供的阶段结果用于验证数值正确性。

2 Overview：方法与数据流程

提供的预处理模块完成缩放、中心裁剪和通道归一化。每张图像切成 7×7 个 32×32 图像块，每块按通道、高、宽顺序展平为 3072 个值，投影至 768 维。前置 CLS token 并加入位置向量后，经过前置归一化和 12 个 Transformer 块；取最终 CLS token，经过后置归一化、投影和长度归一化，得到最终特征。固定权重只用于前向计算；参考数组只用于检查。

预处理按图像短边缩放到 224 像素，再做中心裁剪并转成 RGB。像素转换为 float32 后除以 255，并按通道执行 $(x - \mu)/\sigma$；其中 $\mu = (0.48145466, 0.4578275, 0.40821073)$，$\sigma = (0.26862954, 0.26130258, 0.27577711)$。这些固定常数与裁剪尺寸影响输入 token，因此在编码器之外单独提供并用预处理参考数组验证。

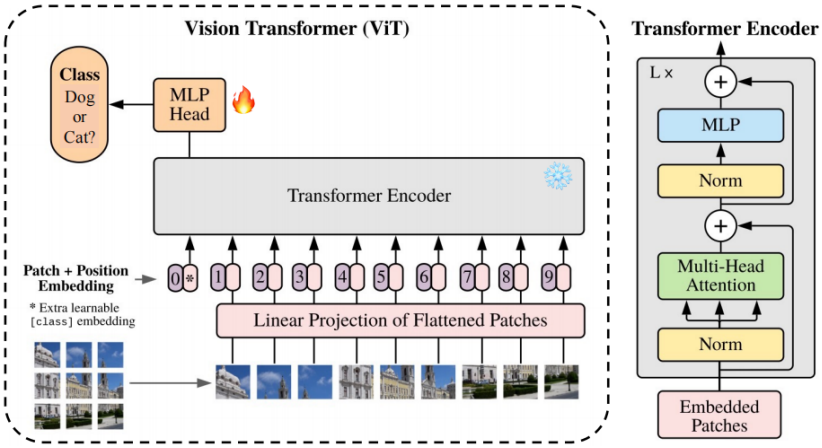


Figure 1: ViT 图像编码流程（课程模板提供）

3 Implementations：关键实现与贡献

图像块和嵌入。 `patchify` 先将输入重排为 $[B, 3, 7, 32, 7, 32]$，再交换网格轴与通道轴，得到 $[B, 7, 7, 3, 32, 32]$，最后展平为 $[B, 49, 3072]$。投影后拼接 CLS 和位置向量，得到 $[B, 50, 768]$。轴顺序决定每块的像素排列。

切块顺序与投影。两个 7 维网格轴分别表示垂直和水平方向；将它们移到通道轴之前，展平时便得到先逐行扫描 patch 网格、再按 C/H/W 展平单个 patch 的顺序。patch 投影为 $X_p W_p^{\mathsf{T}}$，其中 W_p 为 $[768, 3072]$，且没有偏置。将学习到的 CLS 行放在 49 个 patch 行之前，再逐元素加 $[50, 768]$ 位置向量；位置向量通过 batch 轴广播。最终嵌入张量为 $[B, 50, 768]$。

形状检查的作用。 $224/32 = 7$，所以每张图像产生 49 个 patch token；加上 CLS 后恰为 50 个 token。实现时如果把 H/W 网格轴或 C/H/W 顺序弄反，形状仍可能正确，但 patch 内容会错位。因此本项目除检查形状外，也直接将 patch 与嵌入值逐元素同参考比较。

注意力和 Transformer。每个 768 维 token 分成 12 个 64 维头。查询与键相乘并除以 $\sqrt{64}$，经 softmax 加权值向量；合并 12 个头并投影。每块包含注意力残差与 MLP 残差，MLP 使用给定的精确 GELU。最终选择 CLS，乘形状为 $[768, 512]$ 的投影矩阵，并缩放成单位向量。

多头计算中，Q, K, V 的形状均为 $[B, 12, N, 64]$，分数矩阵为 $[B, 12, N, N]$；softmax 沿最后一个键轴计算。注意力输出先恢复成 $[B, N, 768]$，再做线性投影。块内两条残差分别为 $r = x + \mathrm{Attention}(\mathrm{Norm}_1(x))$ 和 $y = r + \mathrm{MLP}(\mathrm{Norm}_2(r))$。这两处相加都保持 token 序列形状 $[B, N, 768]$，使 12 个块能够顺序连接。

本人贡献。本人完成六组 NumPy 练习的实现、README 整理、数值验证和报告撰写。实现过程使用生成式人工智能工具辅助；课程提供的预处理、辅助函数、权重、样例图像及参考检查器作为既有资料使用。

计算流程中的权重方向。Q/K/V 与注意力输出投影、MLP 两个全连接层的参数按 $[\mathrm{out}, \mathrm{in}]$ 保存，故输入右乘其转置并加偏置。最终视觉投影矩阵本身为 $[768, 512]$，按原方向相乘。区分这两种约定能避免结果维度不匹配或数值错误。

4 Results：结果与证据

运行环境为 Python 3.14.3、NumPy 2.4.4、Pillow 12.2.0。执行 `python3 check.py` 加参数 `--stage all` 后全部阶段通过；完整终端记录保存在项目的 `report/check-results.txt`。各阶段与独立 FP32 参考的绝对误差如下：

阶段	输出形状	最大绝对误差	平均绝对误差
预处理	$(10, 3, 224, 224)$	0	0
图像块	$(2, 49, 3072)$	0	0
输入 token	$(2, 50, 768)$	0	0
首个块	$(2, 50, 768)$	7.540×10^{-6}	2.274×10^{-7}
最后 token	$(2, 50, 768)$	4.387×10^{-5}	2.406×10^{-6}
最终输出	$(10, 512)$	6.780×10^{-7}	8.215×10^{-8}
变序及重复批次	$(3, 512)$	4.619×10^{-7}	8.133×10^{-8}

最终输出容差为 $\text{atol} = 2\times10^{-5}, \text{rtol} = 2\times10^{-4}$；内部 token 容差为 $\text{atol} = 2\times10^{-4}, \text{rtol} = 2 \times 10^{-4}$。检查器还要求形状正确、数值有限且类型为 float32。运行 `python3 run.py --output output_tokens.npz` 得到十个 ID 和 $(10, 512)$ 特征；每行的欧氏长度介于 0.9999999 与 1.0 之间。

批次测试将样例 1、0、1 依次输入，既改变顺序又重复第一张所选图像。输出与参考中相同顺序的三行比较仍通过，说明程序没有把固定的参考特征缓存为结果。权重由仓库通过 Git LFS 提供；参考数组只用于比较，生成的 NPZ 特征文件保存在运行目录。项目 GitHub 地址：`https://github.com/xiaogex57/project1-numpy-image-encoder`。

复现步骤。安装 Git LFS 后克隆仓库并运行 `git lfs pull` 获取权重和参考文件；在项目根目录运行 `python3 -m pip install -r requirements.txt` 安装依赖。先用 `python3 check.py --stage assets` 验证资源，再运行 `python3 check.py --stage all`；最后用 `python3 run.py --output output_tokens.npz` 导出结果。参数和样例资源有 SHA-256 清单，完整检查日志随仓库保存。

证据范围。资源检查核对 152 个张量的名称、形状、数据类型和哈希，并验证十张样例图及参考输出顺序。patch、输入 token 和首个 Transformer 块以两张图像逐项对照；完整 12 层输出和最终特征在十张图像上比较。批次用索引 $[1, 0, 1]$ 重新排列并重复样例，检查编码器没有依赖固定输入次序。

5 Discussion：分析与反思

图像块和输入 token 的误差为零，说明轴重排、patch 投影和位置顺序与参考一致。经过 12 个块后内部 token 最大误差为 4.387×10^{-5}，仍在容差内；最终输出最大误差为 6.780×10^{-7}。改变批次顺序并重复图像仍通过，支持按输入行独立处理的结论。

原始项目首次运行检查时，`patches` 阶段因 TODO-01 抛出 `NotImplementedError`；补全图像块排列后该阶段误差降为零。实现中还需注意两种权重方向：大多数线性层权重是 $[\text{out}, \text{in}]$，计算时转置；最终 `visual.proj` 已是 $[768, 512]$，不转置。多头键矩阵也只交换最后两个轴。实验仅验证固定权重的前向数值一致性，不评价分类准确率或训练效果。

数值误差主要出现在注意力矩阵乘法和 MLP 的连续浮点运算后；仅凭误差大小不能判定哪一个算子贡献最大，因此本实验采用逐阶段对照定位偏差。进一步改进可加入不同批大小、非课程样例图像和边界输入，检验接口的通用性；若需要评价实际图像语义能力，还需另设下游任务与指标。

实现取舍与局限。本实现使用 NumPy 的 reshape、transpose 和矩阵乘法表达张量流程，避免显式遍历 patch 或注意力分数。注意力分数张量规模随 BHN^2 增长；MLP 中间维度为 3072，均会影响内存开销。本项目固定图像大小、patch 大小和模型结构，未加入可变分辨率、权重训练或推理加速。结果证明的是给定参数下的数值一致性，不足以证明模型对任意图像的语义判断质量。

反思。本次最直接的调试信号来自阶段边界：原始 patch 检查因缺失实现而抛出异常，补齐轴拆分后 patch 和嵌入参考同时通过；之后误差才出现在 Transformer 运算中。逐阶段验证能把定位范围缩小到最近完成的计算单元。后续若继续扩展，可为输入尺寸、通道数和 dtype 添加显式检查，并用小型手算例子独立核对多头拆分与合并，减少只依赖大型参考张量的风险。

指导教师批阅意见：

成绩评定：

指导教师签字：

年　月　日

备注：报告内的项目或内容设置，可根据实际情况加以调整和补充。